

# Book Tracker

Application de suivi de lecture — Journal personnel de livres

Ce rapport présente la réalisation d'une application web complète de suivi de lectures personnelles, développée avec une stack **Node.js / Express / MongoDB** côté serveur et une interface front-end en **HTML / CSS Tailwind**. L'objectif était de concevoir un journal de lecture permettant d'archiver, de suivre la progression et de gérer une collection de livres de façon élégante et fonctionnelle.

|               |                                                   |
|---------------|---------------------------------------------------|
| Étudiant      | Chaimae El Mounjali                               |
| Date de rendu | 21 mai 2026                                       |
| Technologies  | HTML · Tailwind CSS · Node.js · Express · MongoDB |
| Statut        | ✓ Complété                                        |

## 1. Objectifs du projet

L'exercice consistait à créer une application web de A à Z en intégrant plusieurs briques techniques apprises en cours. Les exigences fonctionnelles étaient les suivantes :

- ◆ Créer un formulaire HTML stylisé (Tailwind CSS) pour l'enregistrement de nouveaux livres.
- ◆ Modéliser un livre avec ses attributs : titre, auteur, pages, statut, prix, format, suggéré par, pages lues et état d'avancement (finished).
- ◆ Implémenter une classe **Book** en JavaScript (module ES6) avec les méthodes *constructor*, *currentlyAt()* et *deleteBook()*.
- ◆ Persister les données dans une base **MongoDB** via Mongoose.
- ◆ Afficher une page de suivi listant les livres avec barre de progression et statistiques globales (livres lus, pages totales).

## 2. Architecture technique

Le projet suit une architecture **MVC légère** (Modèle – Vue – Contrôleur) typique d'une application Express :

| Couche           | Fichier(s)                        | Rôle                                               |
|------------------|-----------------------------------|----------------------------------------------------|
| Modèle           | Book.js (class) + schema Mongoose | Définition des données, validation, logique métier |
| Vue              | index.html                        | Interface utilisateur (Tailwind CSS)               |
| Contrôleur / API | server.js (Express routes)        | CRUD REST : GET, POST, PUT, DELETE                 |
| Base de données  | MongoDB (booktracker > books)     | Persistence des documents livres                   |

### 3. Modèle de données — Schéma du livre

Chaque document *book* stocké dans MongoDB contient les champs suivants. Le champ **finished** est calculé automatiquement : il passe à **1** dès que *pagesRead* atteint *numberOfPages*.

| Champ         | Type          | Contrainte / Valeurs                                                      |
|---------------|---------------|---------------------------------------------------------------------------|
| title         | String        | Requis                                                                    |
| author        | String        | Requis                                                                    |
| numberOfPages | Number        | Requis, > 0                                                               |
| status        | String (Enum) | Read · Re-read · DNF · Currently reading · Returned Unread · Want to read |
| price         | Number        | Optionnel (EUR)                                                           |
| pagesRead     | Number        | $0 \leq \text{pagesRead} \leq \text{numberOfPages}$                       |
| format        | String (Enum) | Print · PDF · Ebook · AudioBook                                           |
| suggestedBy   | String        | Optionnel                                                                 |
| finished      | Number (0/1)  | Défaut : 0 — passe à 1 automatiquement                                    |

### 4. Classe Book (module ES6)

La classe **Book** est définie dans son propre fichier module (*book.js*). Elle encapsule toute la logique métier liée à un livre :

|                                |                                                                                                                                |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>constructor(data)</code> | Initialise tous les attributs du livre. Calcule automatiquement <i>finished</i> si <i>pagesRead</i> === <i>numberOfPages</i> . |
| <code>currentlyAt()</code>     | Retourne le pourcentage de progression de lecture ( <i>pagesRead</i> / <i>numberOfPages</i> × 100), arrondi à l'entier.        |

```
deleteBook(id)
```

Supprime le livre correspondant de la base MongoDB via l'API REST (appel DELETE /api/books/:id).

## 5. Captures d'écran de l'application

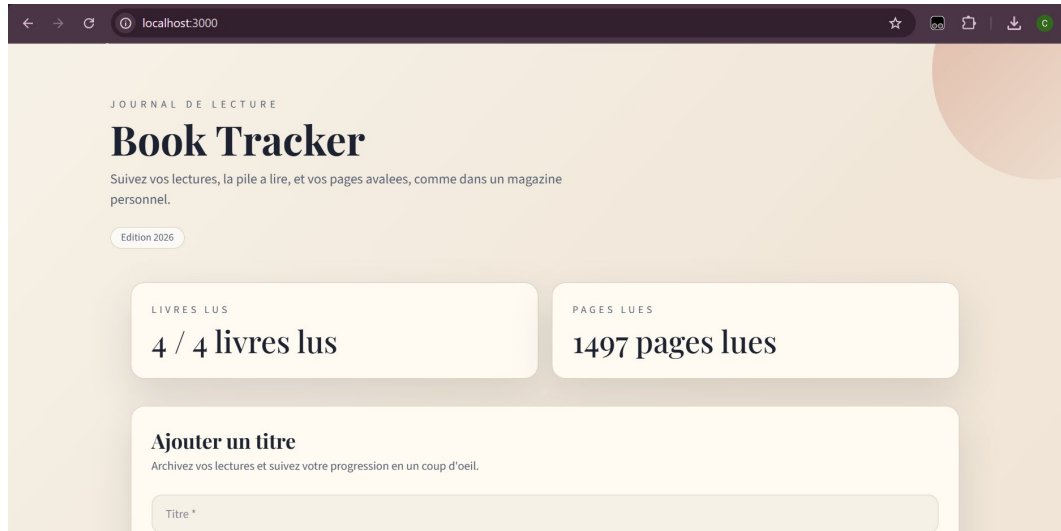


Figure 1 — Page d'accueil : statistiques globales (livres lus, pages lues) et titre éditorial

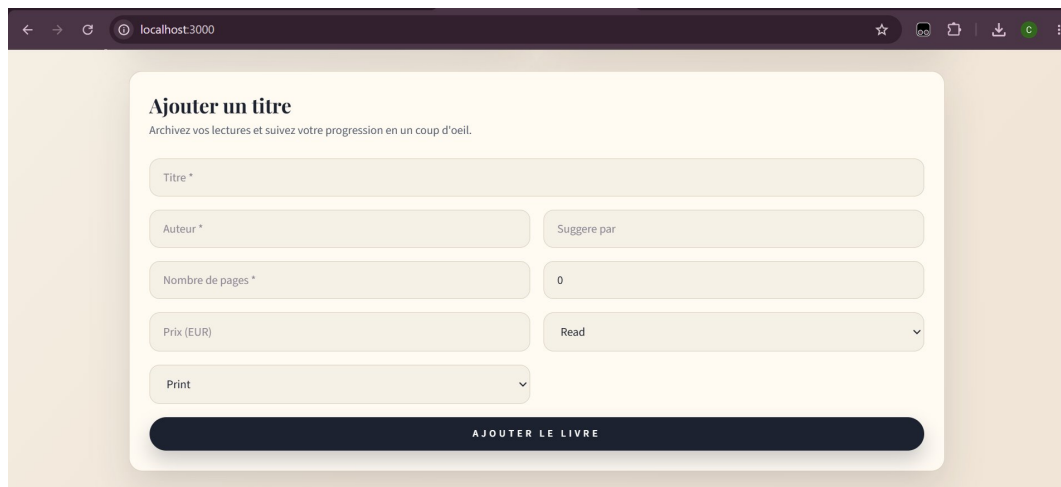


Figure 2 — Formulaire d'ajout d'un livre : champs titre, auteur, pages, statut, format...

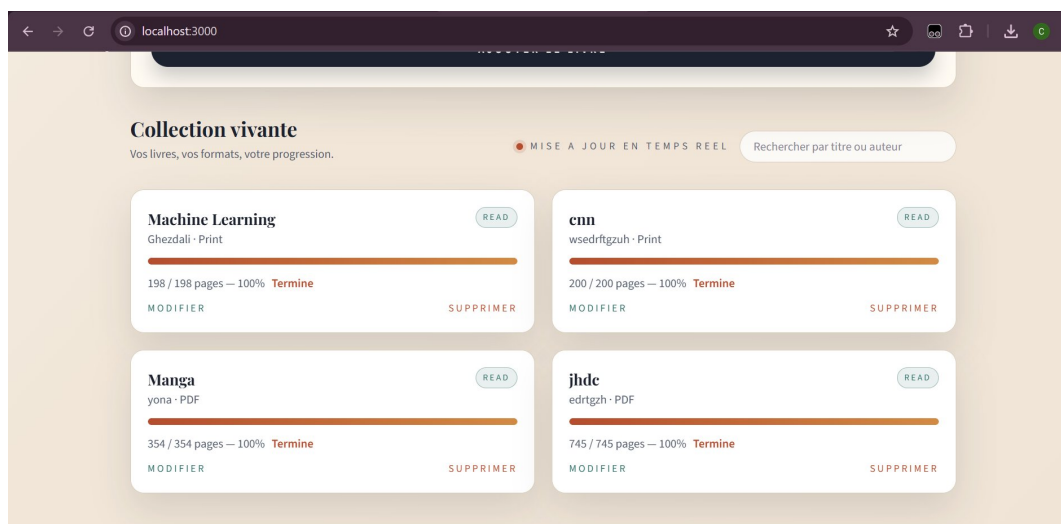


Figure 3 — Collection vivante : cartes de livres avec barres de progression et actions MODIFIER / SUPPRIMER

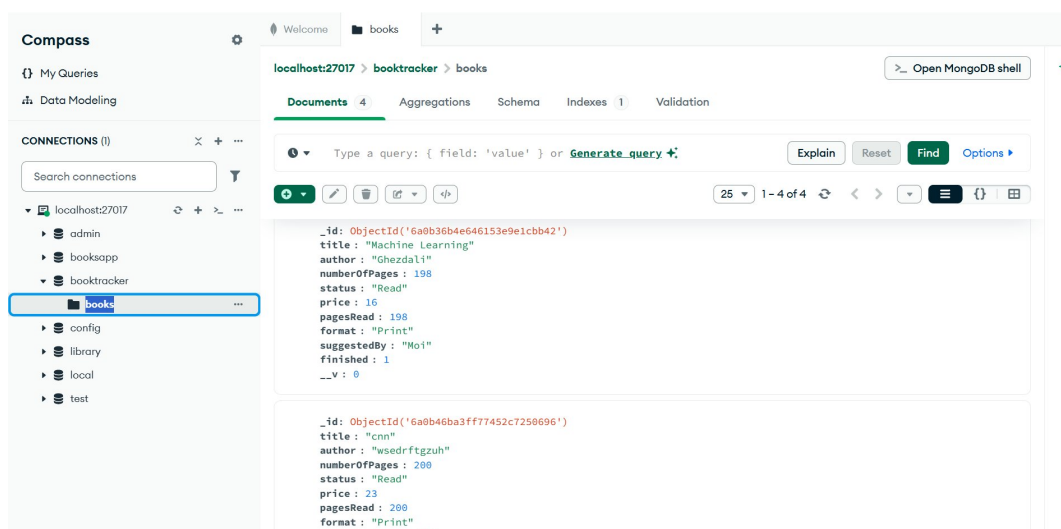


Figure 4 — Base de données MongoDB Compass : collection booktracker > books avec les documents

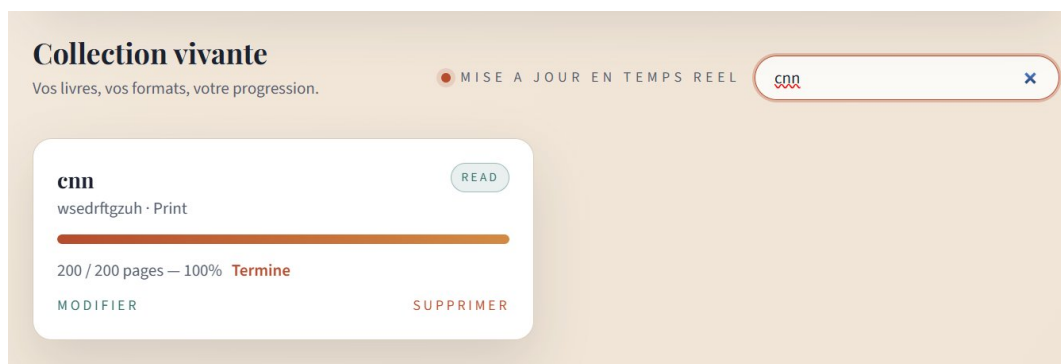


Figure 5 — Recherche en temps réel : filtrage par auteur (ici « cnn »), un seul résultat affiché

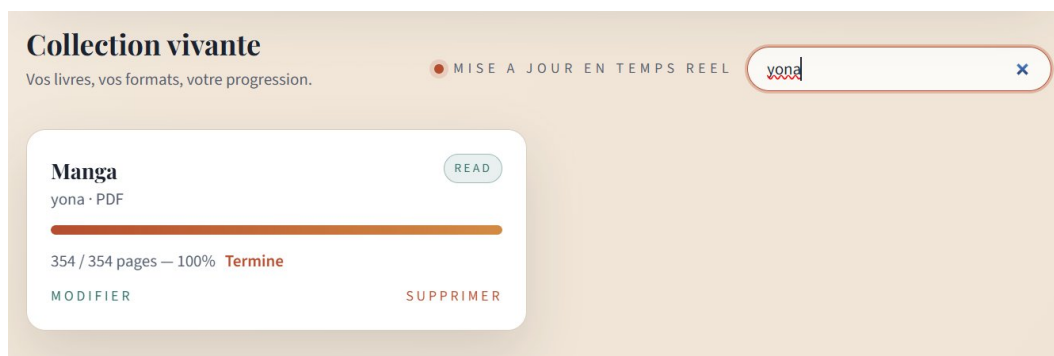


Figure 6 — Recherche par auteur « yona » : affichage du livre Manga (354 pages, 100 %)

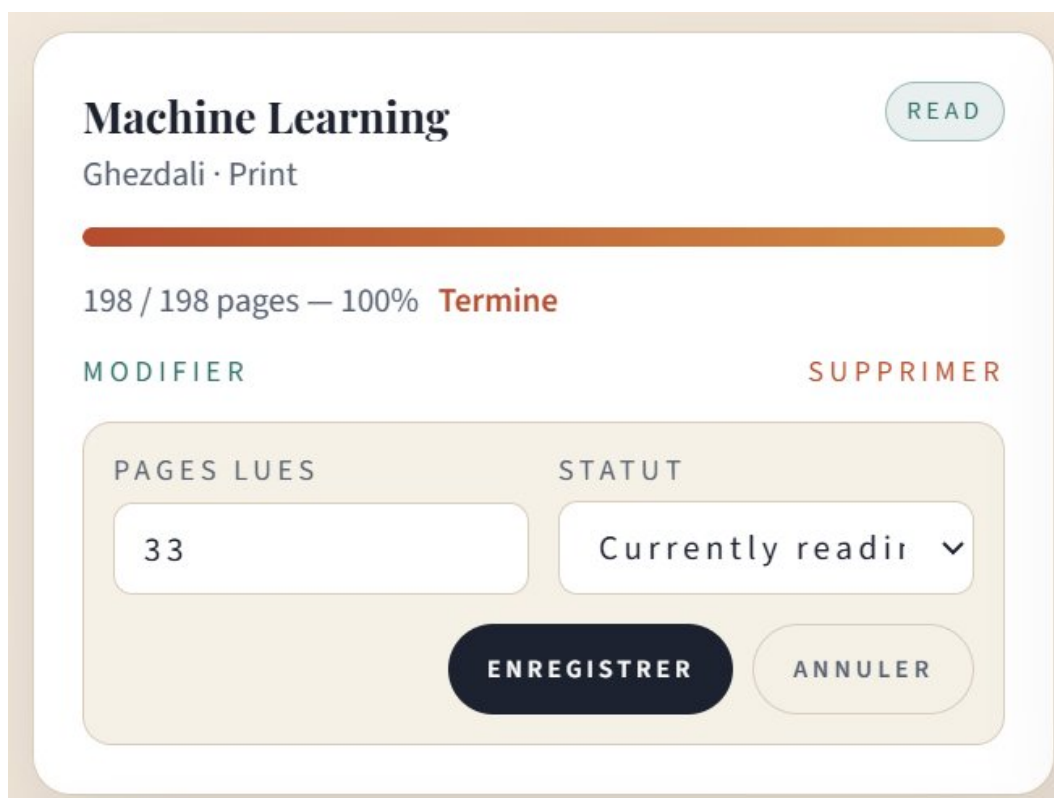


Figure 7 — Mode MODIFIER : formulaire inline pour mettre à jour les pages lues et le statut du livre

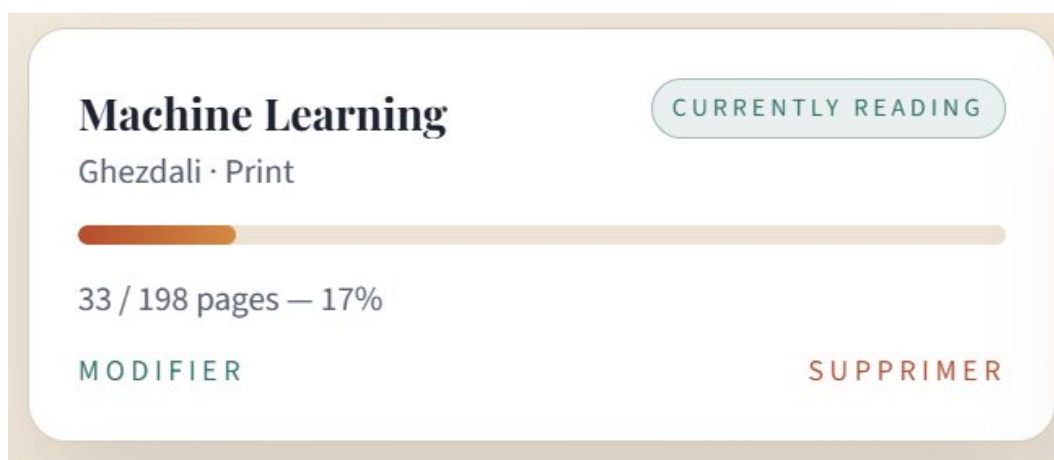


Figure 8 — Après modification : progression mise à jour (33 / 198 pages — 17 %, statut Currently Reading)

## 6. Fonctionnalités implémentées

|                                   |                                                                                                                                                                        |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ajout de livre</b>             | Formulaire complet avec validation côté client et serveur. Les champs obligatoires sont marqués (*). Le champ <i>finished</i> se calcule automatiquement.              |
| <b>Affichage de la collection</b> | Les livres s'affichent sous forme de cartes avec barre de progression colorée (orange → rouge). La mention <b>Terminé</b> apparaît en rouge lorsque le livre est fini. |
| <b>Statistiques globales</b>      | En haut de page : nombre de livres avec statut <i>Read</i> et total de pages lues. Mise à jour en temps réel à chaque modification.                                    |
| <b>Recherche</b>                  | Filtrage instantané de la collection par titre ou par auteur.                                                                                                          |
| <b>Modification</b>               | Bouton MODIFIER sur chaque carte pour mettre à jour les informations (notamment la progression de lecture).                                                            |
| <b>Suppression</b>                | Bouton SUPPRIMER avec confirmation pour retirer un livre de la base.                                                                                                   |
| <b>Persistence MongoDB</b>        | Toutes les données survivent à un redémarrage du serveur grâce à Mongoose et MongoDB local.                                                                            |

---

## 7. Conclusion

Ce projet a permis de mettre en pratique l'ensemble des compétences vues en cours : conception d'une interface responsive avec **Tailwind CSS**, programmation orientée objet en JavaScript (modules ES6), développement d'une API REST avec **Express.js** et persistance des données avec **MongoDB/Mongoose**.

Le résultat est une application fonctionnelle, au design soigné, inspiré d'un magazine personnel. Les quatre livres de test visibles dans MongoDB Compass confirment que toute la chaîne fonctionne : formulaire → API → base de données → affichage.

Pistes d'amélioration envisageables : authentification utilisateur, système de notation par étoiles, export CSV des lectures, ou encore intégration d'une API de livres (Google Books).